



A new library for cross-platform low-level access to USB devices.

nusb is comparable to the C library [libusb](#) and its Rust bindings [rusb](#), but written in pure Rust. It's built on and exposes async APIs by default, but can be made blocking using [futures_lite::future::block_on](#) or similar.

Use nusb to write user-space drivers in Rust for non-standard USB devices or those without kernel support. For devices implementing a standard USB class such as Mass Storage, CDC (Serial), HID, Audio, or Video, this is probably not the library you're looking for – use something built on the existing kernel driver instead. (On some platforms you could detach or replace the kernel driver and program the device from user-space using this library, but you'd have to re-implement the class functionality yourself.)

USB and usage overview

When a USB device connects, the OS queries the device descriptor containing basic device information such as the vendor and product ID (VID / PID) and string descriptors like the manufacturer, product, and serial number strings. [list_devices](#) returns an iterator listing connected USB devices, which can be filtered by these fields to identify and select the desired device.

Call [device_info.open\(\)](#) to open a selected device. Additional information about the device can be queried with [device.active_configuration\(\)](#).

USB devices consist of one or more interfaces exposing a group of functionality. A device with multiple interfaces is known as a composite device. To open an interface, call [Device::claim_interface](#). Only one program (or kernel driver) may claim an interface at a time.

Use the resulting [Interface](#) to transfer data on the device's control, bulk or interrupt endpoints. Transfers are async by default, and can be awaited as individual [Futures](#), or use a [Queue](#) to manage streams of data.

For more details on how USB works, [USB in a Nutshell](#) is a good overview.

Logging

nusb uses the [log](#) crate to log debug and error information.

When [submitting a bug report](#), please include the logs: use a log backend like [env_logger](#) and configure it to enable log output for this crate (for [env_logger](#) set environment variable `RUST_LOG=nusb=debug`.)

Platform support

Linux

nusb is built on the kernel's [usbfs](#) API.

A user must have write access on the `/dev/bus/usb/XXX/YYY` nodes to successfully open a device. Use [udev rules](#) to configure these permissions.

For a single-user system used for development, it may be desirable to give your user account access to all USB devices by placing the following in `/etc/udev/rules.d/70-plugdev-usb.rules`:

```
SUBSYSTEM=="usb", MODE="0660", GROUP="plugdev"
```

This grants access on all USB devices to the `plugdev` group, which your user may be a member of by default on Debian/Ubuntu-based distros. If you are developing an app for others to install, you should scope the permissions more narrowly using the `ATTRS{idVendor}=="ZZZZ", ATTRS{idProduct}=="ZZZZ"` filters to only apply to your device.

Windows

nusb uses [WinUSB](#) on Windows.

On Windows, devices are associated with a particular driver, which persists across connections and reboots. Composite devices appear as multiple devices in the Windows device model, and each interface can be associated with a separate driver.

To use `nusb`, your device or interface must be associated with the WinUSB driver. If you control the device firmware, the recommended way is to use a [WCID](#) descriptor to tell Windows to install the WinUSB driver automatically when the device is first connected. Alternatively [Zadig](#) (GUI) or [libwdi](#) (CLI / C library) can be used to manually install the WinUSB driver for a device.



Rust ▼

nusb uses IOKit on macOS.

Users have access to USB devices by default, with no permission configuration needed. Devices with a kernel driver are not accessible.

Modules

descriptors	Utilities for parsing USB descriptors.
hotplug	Types for receiving notifications when USB devices are connected or disconnected from the system.
transfer	Transfer-related types.

Structs

Device	An opened USB device.
DeviceId	Opaque device identifier
DeviceInfo	Information about a device that can be obtained without opening it.
Interface	An opened interface of a USB device.
InterfaceInfo	Summary information about a device's interface, available before opening a device.

Enums

Speed	USB connection speed
-------	----------------------

Functions

list_devices	Get an iterator listing the connected devices.
watch_devices	Get a Stream that yields an event when a USB device is connected or disconnected from the system.

Type Aliases

Error	OS error returned from operations other than transfers.
-------	---